

Java Method Overriding

1. What is Overriding?

Whatever method a parent class has, is, by default, available to the child class through inheritance. If the child class is **not satisfied** with the parent class's implementation of that method, then the child class is **allowed to redefine** that method based on its own requirement.

This process is called **Overriding**.

Real-time analogy for class:

"Think of a company's HR policy — the head office (parent) sets a general leave policy for all branches. But a branch (child) in a different city, due to local requirements, redefines its own leave policy to suit local needs — while still keeping the same policy 'name'. That's overriding: same method name, new implementation."

Key Terminology

Term	Meaning
Overridden method	The parent class method (the <i>original</i> one that gets overwritten)
Overriding method	The child class method (the <i>new</i> one that redefines it)

💡 **Memory trick:** "Over-*ridden*" = the one that got *ridden over* (parent's old method). "Over-*riding*" = the one *doing the riding* (child's new method).

2. How Does Method Overriding Work Internally?

In method overriding, **method resolution** is taken care of by the **JVM**, based on the **runtime object** (the actual object created at runtime, not the reference type).

Hence, overriding is also considered:

- Runtime Polymorphism
- Dynamic Polymorphism
- Late Binding

The compiler doesn't decide which method to call at compile time — it's decided at **runtime**, depending on which object (parent's or child's) is actually running.

```
class Parent {
    void show() {
        System.out.println("Parent's show() method");
    }
}

class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child's show() method");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent obj = new Child(); // Parent reference, Child object
        obj.show();                // Output: Child's show() method
    }
}
```

1. Compiler (Early Binding)

Checks Reference Type (Left Side)

```
Parent obj = new Child();
```



Does Parent have show()?

✓ Yes (Syntax OK)

2. JVM (Late Binding)

Checks Actual Object (Right Side)

```
Parent obj = new Child();
```



What object is in memory?

→ Executes Child's show()

3. The 5 Golden Rules for Overriding

Rule 1: Method Name and Argument Must Match

The method name and the parameter list (argument types, number, and order) must be **exactly the same** in both parent and child class. (The method signature must be the same).

Rule 2: Return Type Rule (Covariant Return Type)

- **Until Java 1.4:** The return type must be exactly the same.
- **From Java 1.5+:** The return type is allowed to be **covariant** — meaning the child's return type can be a **subclass** of the parent's return type.

Parent Class

```
Product getObject() {  
    return new Product();  
}
```

Child Class (Java 1.5+)

```
@Override  
Laptop getObject() {  
    return new Laptop();  
}
```

↓

✓ Valid! (Laptop IS-A Product)

Rule 3: Cannot Override a final Method

We **cannot override** a parent class's `final` method. `final` means it cannot be changed. If we try, we get a compile-time error.

Rule 4: Access Modifier Rule

The access modifier of the overriding method (child) **must be the same or broader** than the parent's method — it cannot be more restrictive.

PRIVATE

DEFAULT

PROTECTED

PUBLIC

✗ Illegal Override

Parent is **Protected**

↓

Child tries to make it **Private**

Compile Error: Cannot narrow!

✓ Legal Override

Parent is **Default**

↓

Child makes it **Public**

Compiles OK: Broader is fine!

Rule 5: Static Method Rule

We **cannot override a static method as a non-static method** (and vice versa) — it results in a compile-time error. Static methods belong to the class and are resolved at compile time (Method Hiding), whereas overriding depends on runtime objects.

4. Summary Table

Rule #	Rule	Violation Result
1	Method name & parameters must match exactly	Becomes overloading, not overriding
2	Return type must be same (or covariant from 1.5+)	Compile-time error
3	Cannot override a <code>final</code> method	Compile-time error
4	Access modifier must be same or broader	Compile-time error
5	Cannot override <code>static</code> as <code>non-static</code> (or vice versa)	Compile-time error